Instituto Tecnológico de Costa Rica

Centro Académico de Alajuela

Ingeniería en Computación

Asignación de horarios universitarios

mediante programación declarativa y CLP(FD)

Informe técnico y manual de implementación en GNU Prolog

Estudiante: Kendal Josué Chacón Chaves

Curso: [Nombre del curso]

Profesor: Eddy Ramírez Jiménez

II semestre de 2026

Alajuela, Costa Rica

Octubre de 2026

Índice

1. Introducción y Descripción General **1**

1.1. Objetivos del Sistema 1

1.2. Arquitectura Modular del Software 1

2. Modelo Formal del Problema y Representación Interna **4**

2.1. Representación Interna en la Base de Hechos 4

2.2. Dominio Unario y Poda Previa 4

3. Especificación de Restricciones **5**

3.1. Restricciones Duras (Hard Constraints) 5

3.1.1. Restricciones Unarias 5

3.1.2. Restricciones Binarias de No Solapamiento 6

3.2. Restricciones Blandas y Función de Costo 6

4. Estrategias de Resolución y Búsqueda **7**

4.1. Backtracking Puro y con Verificación Anticipada 7

4.2. Programación Lógica con Restricciones (CLP(FD)) 8

4.3. Optimización Mediante Branch and Bound 8

5. Heurísticas de Selección de Variable y de Valor **9**

5.1. Costo Computacional vs. Eficacia de Poda 10

6. Técnicas de Reducción de Simetría **10**

6.1. Simetría 1: Orden Cronológico Intra-Grupo 10

6.2. Simetría 2: Primer Uso de Aulas Equivalentes 10

7. Lectura, Validación y Verificación Independiente **11**

7.1. Parser a Bajo Nivel en GNU Prolog 11

7.2. Validación Semántica en Dos Pasadas 11
7.3. Verificación Independiente (`verifica/1`) 11

8. Diagnóstico y Tolerancia a Fallos **12**
8.1. Algoritmo de Diagnóstico Causal de Inviabilidad 12

9. Evaluación Experimental del Rendimiento **13**
9.1. Metodología y Banco de Pruebas 13
9.2. Resultados Experimentales Consolidados 13
9.3. Análisis Detallado de Aceleración y Eficiencia de Poda 15

10. Extensiones Implementadas **16**

11. Manual de Ejecución y Reproducción **17**
11.1. Requisitos e Instalación 17
11.2. Invocación por Línea de Comandos 17
11.3. Modo Interactivo en GNU Prolog 17
11.4. Compilación a Código Nativo 17

12. Conclusiones y Trabajo Futuro **18**
12.1. Conclusiones 18
12.2. Limitaciones del Sistema 18

A. Ejemplo Completo de Horario Generado **20**

B. Matriz de Cobertura de Pruebas Unitarias **21**

1. Introducción y Descripción General

El problema de asignación de horarios universitarios (*University Course Timetabling Problem*) constituye uno de los desafíos combinatorios más representativos y complejos en la investigación de operaciones y la informática teórica. Clasificado formalmente como $\mathcal{NP}$-duro en sentido fuerte, su resolución exige determinar la asignación de un conjunto de eventos académicos (sesiones de clase) a una rejilla discreta de recursos espaciales (aulas) y temporales (días y franjas horarias), garantizando la no colisión de recursos físicos y humanos, así como el cumplimiento de preferencias pedagógicas y organizativas.

1.1. Objetivos del Sistema

De acuerdo con las especificaciones establecidas en el pliego de requerimientos del proyecto (`logicos.pdf`), el sistema persigue los siguientes objetivos primordiales:

a) **Implementación Pura en GNU Prolog:** Desarrollar el sistema íntegramente en GNU Prolog (versión 1.5.0), sin recurrir a dependencias externas, librerías privativas ni preprocesadores externos, empleando guiones Bash únicamente como envoltorios de invocación.

b) **Dualidad de Motores de Búsqueda:** Implementar de forma paralela y modular dos estrategias de resolución: búsqueda cronológica basada en retroceso (*Backtracking* clásico y anticipado) y programación con restricciones sobre dominios finitos (CLP(FD)).

c) **Satisfacción y Optimización:** Satisfacer estrictamente todas las restricciones duras del dominio e incorporar un motor de optimización exacta mediante ramificación y acotamiento (*Branch and Bound*) para minimizar las violaciones de restricciones blandas.

d) **Diagnóstico y Robustez:** Distinguir con precisión matemática entre fallos sintácticos de entrada, instancias estructuralmente inviables, instancias no resolubles por contradicciones internas y paradas por agotamiento del límite de exploración.

e) **Validación Independiente:** Incorporar un predicado de verificación desacoplado de los mecanismos de búsqueda, capaz de auditar la corrección de cualquier horario propuesto.

1.2. Arquitectura Modular del Software

El sistema se organiza en cinco módulos Prolog claramente diferenciados, garantizando alta cohesión y bajo acoplamiento:

- **`entrada.pl`:** Responsable de la lectura byte a byte, análisis léxico, decodificación UTF-8, validación semántica en dos pasadas y generación de la base de hechos dinámica.
- **`modelo.pl`:** Define los dominios unarios de cada sesión, la compatibilidad entre pares de asignaciones, el cálculo de métricas (huecos de grupo y docente, costos de penalización) y la rutina de verificación independiente (`verifica/1`).
- **`busqueda.pl`:** Contiene los motores de resolución (BT puro, BT con *forward checking*, CLP(FD) y Branch and Bound), las heurísticas de selección de variables y valores, las reducciones de simetría y el algoritmo de diagnóstico causal.
- **`salida.pl`:** Genera los informes formateados en texto plano con rejillas ASCII, la exportación de hechos relacionales consultables en Prolog (`asignacion/7`) y la exportación estructurada en CSV.
- **`cli.pl` y `horarios.pl`:** Coordinador de la interfaz de usuario por línea de comandos, parseo de banderas CLI, control de tiempos de CPU y de pared, y gestión de códigos de retorno del sistema operativo.

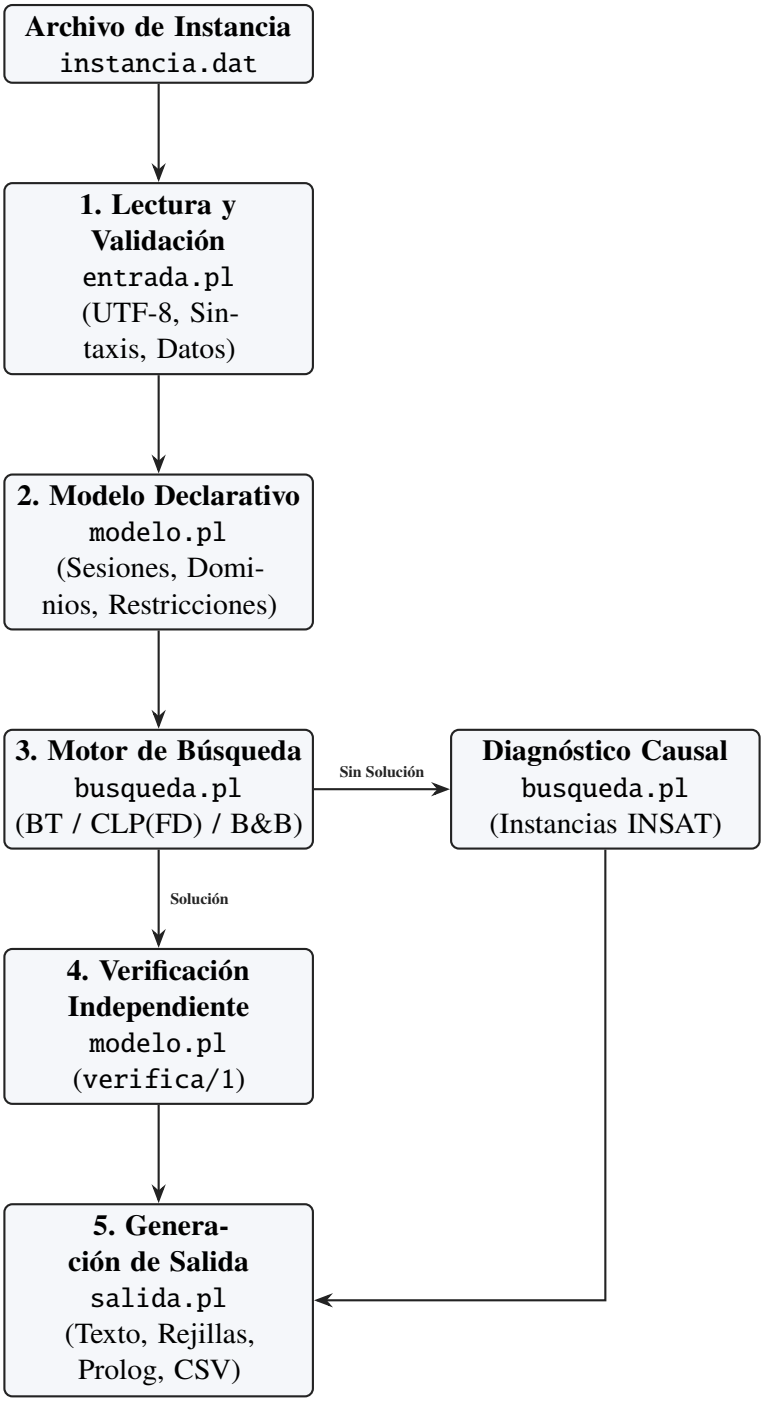


Figura 1: Diagrama de flujo arquitectónico del sistema de asignación de horarios.

2. Modelo Formal del Problema y Representación Interna

2.1. Representación Interna en la Base de Hechos

Tras la fase de lectura y validación, la instancia se afirma dinámicamente en la base de conocimientos de GNU Prolog mediante términos canónicos que conservan fielmente la capitalización y caracteres especiales:

```
config(dias, ['L','M','X','J','V']).
config(franjas, 7).
config(inicio, 420).            % 07:00 en minutos transcurridos desde
   medianoche
config(duracion_franja, 60).     % minutos por franja
aula('A-101', 45, teoria).
profesor('P-001', 'Ana Rojas', 12, [slot('L',1), slot('L',2), ...]).
grupo('IC-1802', 'C1', 'Introduccion a la Programacion', 35, 2, 2, teoria).
imparte('IC-1802', 'C1', 'P-001').
restriccion(aula_fija('IC-2101','C1','LAB-1'), 42).
restriccion(prohibida('MA-1002','C1','V',5), 43).
restriccion(prefiere('IC-1802','C1','L',1,3), 44).
restriccion(compacta('IC-1802','C1',5), 45).
```

Listing 1: Estructura de la base de conocimientos dinámica.

Nótese que el último argumento de `restriccion/2` conserva el número de línea de origen en el archivo de texto, lo que resulta indispensable para la generación precisa del bloque de diagnóstico de inviabilidad.

Una asignación completa se modela mediante el término compuesto:

```
asig(Curso, Grupo, NumeroSesion, Aula, Dia, FranjaInicio, Duracion)
```

mientras que las sesiones abstractas se identifican unívocamente mediante `sesion(Curso, Grupo, NumeroSesion)`.

2.2. Dominio Unario y Poda Previa

El predicado declarativo `dominio(+Sesion, -Valores)` calcula extensionalmente el conjunto de tuplas admisibles para una sesión dada, aplicando de forma estricta las restricciones unarias antes de inicializar cualquier mecanismo de búsqueda:

```
dominio(sesion(C,G,I), Valores) :-
    grupo(C,G,_,Ins,N,Dur,Tipo), mod_entre(1,N,I),
    config(dias,Dias), config(franjas,Fmax), Ult is Fmax-Dur+1,
    findall(asig(C,G,I,A,D,F,Dur),
      ( aula(A,Cap,TA), Cap >= Ins, tipo_compatible(Tipo,TA),
        mod_miembro(D,Dias), mod_entre(1,Ult,F),
        mod_profesores_disponibles(C,G,D,F),
        \+ (restriccion(aula_fija(C,G,AF),_), A \== AF),
        Fin is F+Dur-1,
        \+ (restriccion(prohibida(C,G,D,FP),_), FP >= F, FP =< Fin)
      ), Valores).
```

Listing 2: Cálculo del dominio unario para una sesión.

Esta construcción garantiza que ninguna tupla generada viole la capacidad del aula, la compatibilidad de tipo, los límites de inicio para no desbordar el día lectivo, la disponibilidad del docente en el momento del inicio, las aulas fijas ni las franjas explícitamente prohibidas.

3. Especificación de Restricciones

3.1. Restricciones Duras (Hard Constraints)

Las restricciones duras son de obligado cumplimiento; cualquier asignación que viole una sola restricción dura es considerada infactible y desechada por el sistema.

3.1.1. Restricciones Unarias

1. **Capacidad de Aula:** $\text{cap}(a_s) \geq \text{ins}(c_s, g_s)$.
2. **Tipo de Aula:** Compatibilidad semántica entre el requerimiento del curso y el aula asignada:

$$\begin{aligned} \text{teoria} &\implies \text{tipo}(a_s) \in \{\text{teoria}, \text{mixta}\} \\ \text{laboratorio} &\implies \text{tipo}(a_s) \in \{\text{laboratorio}, \text{mixta}\} \\ \text{mixta} &\implies \text{tipo}(a_s) \in \{\text{teoria}, \text{laboratorio}, \text{mixta}\} \end{aligned}$$

3. **Confinamiento Diario:** La sesión debe completarse íntegramente dentro del día lectivo asignado: $f_s + L(s) - 1 \leq |F|$.

4. **Disponibilidad Docente:** En la franja de inicio f_s, todos los profesores asignados al grupo deben tener disponibilidad declarada: $(d_s, f_s) \in \text{disp}(p), \forall p \in \text{imparte}(c_s, g_s)$.
5. **Aula Fija:** Si existe una restricción declarada `aula_fija(c, g, A)`, entonces $a_s = A$.
6. **Franja Prohibida:** Si existe `prohibida(c, g, D, F)`, el intervalo de la sesión $[f_s, f_s + L(s) - 1]$ no puede contener la franja prohibida en el día D:

$$d_s = D \implies F \notin [f_s, f_s + L(s) - 1] \tag{1}$$

7. **Carga Semanal Máxima del Docente:** La suma de las duraciones de todas las sesiones asignadas a un docente no puede exceder su límite:

$$\forall p \in P, \quad \sum_{(c,g)|p \in \text{imparte}(c,g)} N(c,g) \cdot L(c,g) \leq \text{max_franjas}(p) \tag{2}$$

3.1.2. Restricciones Binarias de No Solapamiento

Dos sesiones distintas $s_1, s_2 \in S$ coinciden temporalmente si y solo si se imparten el mismo día ($d_{s_1} = d_{s_2}$) y sus intervalos de franjas semiabiertos $[f_{s_1}, f_{s_1} + L(s_1))$ y $[f_{s_2}, f_{s_2} + L(s_2))$ se intersecan:

$$\text{solapan}(s_1, s_2) \iff (d_{s_1} = d_{s_2}) \wedge (f_{s_1} < f_{s_2} + L(s_2)) \wedge (f_{s_2} < f_{s_1} + L(s_1)) \tag{3}$$

Bajo condición de solapamiento temporal, el sistema prohíbe terminantemente:

1. **Solapamiento de Grupo:** $(c_{s_1} = c_{s_2}) \wedge (g_{s_1} = g_{s_2}) \implies \neg\text{solapan}(s_1, s_2)$.
2. **Solapamiento de Aula:** $a_{s_1} = a_{s_2} \implies \neg\text{solapan}(s_1, s_2)$.
3. **Solapamiento de Docente:** $\big(\text{imparte}(c_{s_1}, g_{s_1}) \cap \text{imparte}(c_{s_2}, g_{s_2}) \neq \emptyset\big) \implies \neg\text{solapan}(s_1, s_2)$.
4. **Grupos Excluyentes:** Si se declaró `excluyentes(`c_1`, `g_1`, `c_2`, `g_2`)`, ambos grupos no pueden solaparse en franja alguna.

3.2. Restricciones Blandas y Función de Costo

Las restricciones blandas cuantifican la calidad del horario generado. Pueden ser violadas, pero cada transgresión impone una penalización proporcional a su peso $w \in \mathbb{N}^+$.

- **Preferencia Horaria (`prefiere`):** Declara una penalización w si el grupo (c, g) no cuenta con ninguna sesión cuyo inicio coincida exactamente con (D, F).

- **Compactación de Horario (`compacta`):** Penaliza la presencia de franjas libres internas (huecos) entre sesiones de un mismo grupo dentro de una misma jornada. Para un día $d \in D$, si las franjas ocupadas por el grupo son $F_{c,g,d} \neq \emptyset$, con franja mínima $f_{\text{mín}}$ y franja máxima $f_{\text{máx}}$, la cantidad de huecos se define como:

$$\text{huecos}(c, g, d) = (f_{\text{máx}} - f_{\text{mín}} + 1) - |F_{c,g,d}| \tag{4}$$

  La penalización agregada es:

$$\text{costo}_{\text{comp}}(c, g, w) = w \cdot \sum_{d \in D} \text{huecos}(c, g, d) \tag{5}$$

El costo total de un horario H corresponde a la suma ponderada de todas las restricciones blandas activas:

$$\text{costo}(H) = \sum_{r_k \in \mathcal{R}_{\text{blandas}}} w_k \cdot \text{violaciones}(r_k, H) \tag{6}$$

4. Estrategias de Resolución y Búsqueda

El proyecto implementa y compara experimentalmente dos paradigmas completos de búsqueda combinatoria sobre el motor de GNU Prolog.

4.1. Backtracking Puro y con Verificación Anticipada

El motor de retroceso (`bt`) opera asignando recursivamente una tupla de su dominio unario a cada sesión pendiente. Se implementan dos variantes operativas:

1. **Generación y Prueba (*Generate and Test*):** En su modalidad más ingenua (`-sin-anticipada`), el motor instancia consecutivamente todas las sesiones a partir de sus dominios unarios estáticos. Una vez completado el horario, se invoca el verificador binario sobre el conjunto total. Si existe colisión, se retrocede en cascada.

2. **Verificación Anticipada (*Forward Checking*):** En la modalidad recomendada (`-anticipada`), tras cada asignación parcial de una sesión s_i, se verifica de inmediato la compatibilidad con todas las sesiones ya asignadas. Adicionalmente, se filtran los dominios de todas las sesiones pendientes $s_j \in S_{\text{restantes}}$, eliminando cualquier candidato que entre en colisión con la decisión recién tomada. Si el dominio de alguna sesión futura queda vacío ($D_j = \emptyset$), la rama se descarta inmediatamente mediante fallo (`fail`), evitando explorar subárboles condenados al fracaso.

4.2. Programación Lógica con Restricciones (CLP(FD))

La estrategia basada en dominios finitos (`clpfd`) explota el resolutor nativo de GNU Prolog mediante una transformación ingeniosa de las decisiones multidimensionales a índices escalares enteros:

1. **Variable de Decisión Discreta:** Cada sesión s_i se asocia con una variable FD $V_i \in [1, |Dom(s_i)|]$, creada mediante `fd_domain(Vi, 1, Ni)`.

2. **Canalización Espaciotemporal (*Channeling*):** Para evitar construir tablas cuadráticas extensionales, se extraen dos listas de constantes paralelas a partir del dominio unario: los tiempos de inicio absolutos ($T = \text{índice_día} \cdot |F| + \text{franja}$) y los índices ordinales de aula ($A \in [1, |A|]$). Mediante la primitiva `fd_element/3`, se vincula la variable índice V_i con las variables enteras continuas de tiempo T_i y aula A_i:

```
    fd_element(Vi, Inicios, Ti),
    fd_element(Vi, Aulas, Ai)

```

3. **Postulación de Restricciones Disyuntivas:** Si dos sesiones s_1, s_2 comparten grupo, profesor o una exclusión declarada, deben separarse temporalmente en el tiempo global:

```
    (T1 + Dur1 #=< T2) #\/ (T2 + Dur2 #=< T1)

```

   En caso de no compartir dichos recursos, pueden coincidir en el tiempo siempre y cuando ocupen aulas distintas:

```
    (T1 + Dur1 #=< T2) #\/ (T2 + Dur2 #=< T1) #\/ (A1 #\= A2)

```

4. **Refuerzo Redundante con `fd_all_different/1`:** Se aplica `fd_all_different` sobre el conjunto de variables $\{T_{c,g,1}, \ldots, T_{c,g,k}\}$ correspondientes a los inicios absolutos de las sesiones del mismo grupo. Esto intensifica la propagación de consistencia de arco sin generar falsos positivos.

4.3. Optimización Mediante Branch and Bound

Para resolver el problema de optimización bajo la bandera `-optimizar`, el sistema implementa una búsqueda con ramificación y acotamiento (*Branch and Bound*) controlada sobre el árbol de decisiones:

- Se almacena dinámicamente el mejor costo encontrado hasta el momento C_{mejor} mediante el hecho `bus_mejor(Horario, Costo)`.
- En cada nodo del árbol (tanto en BT como en CLP(FD)), se calcula una **cota inferior admisible** $LB(\text{Asignadas}, \text{Pendientes})$ a partir de las penalizaciones de preferencia (`prefiere`):

$$LB = \sum_{(c,g,D,F,w) \in \mathcal{P}_{\text{imposibles}}} w \tag{7}$$

  donde una preferencia se declara *imposible* si ninguna sesión asignada la cumple y ningún candidato restante en los dominios de las tareas pendientes posee inicio en (D, F).
- **Poda por Cota:** Si en cualquier punto de la exploración se determina que $LB \geq C_{\text{mejor}}$, la rama completa se poda inmediatamente.

5. Heurísticas de Selección de Variable y de Valor

El orden de exploración del espacio de estados determina críticamente el tamaño del árbol generado. Se implementan y parametrizan las siguientes heurísticas:

Heurística	Tipo	Criterio de Ordenamiento y Justificación
`original`	Variable	Orden estático de lectura de los grupos en el archivo.
`mrv` / `ff`	Variable	*Minimum Remaining Values* / *First-Fail*: Selecciona la sesión con el menor número de valores admisibles en su dominio actual ($\|Dom(s)\|$). Acelera la detección temprana de fallos.
`ffc`	Variable	Aplica MRV, y ante empates, prioriza la sesión con mayor grado de colisión con otras tareas.
`grado`	Variable	Prioriza sesiones que presentan mayor cantidad de vecinos con conflictos potenciales en sus dominios.
`demanda`	Variable	Orden estático descendente por el producto: inscritos × semanales × duracion. Ubica primero las sesiones más difíciles de acomodar.
`original`	Valor	Orden canónico del dominio: aula, día y franja de lectura.
`lcv`	Valor	*Least Constraining Value*: Ordena los valores candidatos evaluando el impacto en las sesiones vecinas, eligiendo primero aquel que elimina la menor cantidad de candidatos en las tareas restantes.

Tabla 1: Heurísticas de selección de variables y valores implementadas.

5.1. Costo Computacional vs. Eficacia de Poda

El análisis experimental revela que heurísticas sofisticadas como `grado` y `lcv` conllevan una sobrecarga computacional de orden $O(|S| \cdot |Dom|)$ en cada decisión. En instancias con restricciones holgadas, el tiempo invertido en inspeccionar pares de listas puede sobrepasar con creces el ahorro obtenido en nodos podados. En contraste, `mrv` (o `ff` en CLP(FD)) ofrece una relación óptima entre costo de inspección y reducción del árbol.

6. Técnicas de Reducción de Simetría

La presencia de soluciones isomorfas genera ramas redundantes en el árbol de búsqueda. El sistema incorpora dos técnicas formales de reducción de simetría (activadas por defecto y desactivables mediante `-sin-simetria`):

6.1. Simetría 1: Orden Cronológico Intra-Grupo

Para un mismo grupo (c, g), sus N sesiones semanales son idénticas e intercambiables en cuanto a requerimientos y restricciones duras. Permutar los índices internos de sesión produce horarios idénticos en la práctica.

$$\forall i, j \in \{1, \ldots, N(c, g)\}, \quad i < j \implies T(s_{c,g,i}) < T(s_{c,g,j}) \tag{8}$$

En CLP(FD), esta restricción se impone directamente mediante la desigualdad reificada $T_i < T_j$, eliminando $N!$ permutaciones simétricas para cada grupo.

6.2. Simetría 2: Primer Uso de Aulas Equivalentes

Dos aulas $a_1, a_2 \in A$ se consideran formalmente equivalentes si y solo si poseen la misma capacidad, el mismo tipo y ninguna de ellas aparece referenciada en una directiva de `aula_fija`. Intercambiar globalmente las asignaciones de a_1 y a_2 conserva la validez y el costo del horario.

Para romper esta simetría sin podar soluciones factibles durante exploraciones no cronológicas (como cuando opera MRV), el sistema inspecciona el prefijo continuo de sesiones asignadas en el orden de lectura y exige que un aula equivalente no pueda ser utilizada por primera vez si existe otra aula equivalente precedente en el orden del archivo que aún no haya sido estrenada. La corrección y admisibilidad de ambas reducciones de simetría se valida exhaustivamente mediante el script `tests/simetrias.pl` sobre 16 micromodelos combinatorios independientes.

7. Lectura, Validación y Verificación Independiente

7.1. Parser a Bajo Nivel en GNU Prolog

El módulo `entrada.pl` implementa un autómata de lectura recursivo basado en la primitiva `get_byte/2`. Procesa directamente archivos de texto plano evitando cualquier herramienta externa. El lector:

- Decodifica secuencias UTF-8 multibyte y suprime de manera transparente el *Byte Order Mark* (BOM) (`0xEF, 0xBB, 0xBF`).
- Normaliza finales de línea Unix (`LF`) y Windows/DOS (`CRLF`), reconociendo archivos donde el último registro concluye directamente con el carácter de fin de archivo (`EOF`).
- Recorta espacios en blanco y tabuladores periféricos en cada celda delimitada por punto y coma (`;`).

7.2. Validación Semántica en Dos Pasadas

Para permitir que las secciones (`#:CONFIG`, `#:AULA`, `#:PROFESOR`, `#:CURSO`, `#:IMPARTE`, `#:RESTRICCION`) aparezcan en cualquier orden sin dependencias cíclicas, la validación se estructura en dos fases:

1. **Validación Léxica y Sintáctica Local:** Verifica aridades, números enteros no negativos, obligatoriedad de campos y ausencia de registros huérfanos antes del primer encabezado de sección.
2. **Validación de Integridad Referencial:** Comprueba la existencia de claves `CONFIG`, que las asignaciones en `#:IMPARTE` y `#:RESTRICCION` apunten a cursos, grupos, profesores y aulas efectivamente declarados, y que los códigos de día y franja pertenezcan al dominio configurado.

Los errores no provocan abortos abruptos; se acumulan en una lista descriptiva indicando el número exacto de línea de cada transgresión.

7.3. Verificación Independiente (`verifica/1`)

Para garantizar la integridad científica de los resultados y evitar sesgos derivados de posibles errores en los motores de búsqueda, el predicado `verifica/1` opera de forma 100 % desacoplada:

- **No** invoca a `dominio/2` ni a `compatible_asignaciones/2`.
- Comprueba de nuevo que el horario sea una lista cerrada (*ground*) con exactamente la cantidad de sesiones requerida por el modelo.
- Reexamina todas las restricciones unarias y compara cuadráticamente cada par de asignaciones para detectar colisiones inadvertidas de aula, docente, grupo o exclusión.
- Audita la carga acumulada de cada docente frente a su límite semanal.

Todo horario encontrado debe superar `verifica/1` antes de considerarse una solución válida y proceder a su exportación.

8. Diagnóstico y Tolerancia a Fallos

El sistema clasifica el resultado de cualquier ejecución en exactamente cuatro estados mutuamente excluyentes, mapeados a códigos de retorno estándar del sistema operativo (`exit status`):

Estado Emitido	Código CLI	Salida	Causa Determinante
`SOLUCION ENCONTRADA`	0	Completa	Se encontró una asignación que satisface todas las restricciones duras.
`INSTANCIA INVALIDA`	2	Error	Archivo inexistente, sintaxis corrupta o inconsistencia previa evidente.
`LIMITE ALCANZADO`	3	Encabezado	Se alcanzó la cuota máxima de nodos sin completar el horario (no genera horario parcial corrupto).
`SIN SOLUCION`	4	Diagnóstico	Búsqueda exhaustiva concluida sin éxito; la instancia es formalmente insatisfacible.

Tabla 2: Estados finales de ejecución y códigos de retorno del sistema.

8.1. Algoritmo de Diagnóstico Causal de Inviabilidad

Ante una instancia con sintaxis válida pero matemáticamente imposible (`sin-solucion.dat`), el sistema activa el motor de diagnóstico causal acotado:

1. Resuelve primero la instancia desactivando temporalmente todas las restricciones duras declaradas (`aula_fija`, `prohibida`, `excluyentes`). Si ya es imposible, señala una incompatibilidad fundamental de recursos básicos.

2. En caso de ser resoluble la base, incorpora las restricciones declaradas secuencialmente en el orden de lectura $\{r_1, r_2, \ldots, r_k\}$, ejecutando una búsqueda rápida con límite de 10 000 nodos.
3. La primera restricción r_m cuyo prefijo $\{r_1, \ldots, r_m\}$ se demuestra insatisfacible se reporta formalmente en el bloque `DIAGNOSTICO` como la causa primaria de la inviabilidad.
4. Adicionalmente, se cuantifica el impacto individual de cada restricción midiendo la reducción porcentual de candidatos unarios tras la propagación inicial.

9. Evaluación Experimental del Rendimiento

9.1. Metodología y Banco de Pruebas

Las pruebas experimentales se ejecutaron en un entorno Linux x86_64 bajo GNU Prolog 1.5.0, asignando 64 MB de pila local y 128 MB de pila global (`LOCALSZ=65536`, `GLOBALSZ=131072`). Todas las mediciones se registraron mediante `statistics(runtime, _)` para tiempo de CPU y `statistics(real_time, _)` para tiempo de pared, integrados en el arnés automatizado `experimentos.pl`.

Se evaluaron 35 combinaciones conformadas por 5 instancias de complejidad creciente y 7 configuraciones algorítmicas, con un corte estricto de **5000 nodos**:

- **Instancias:** `pequena` (3 sesiones, 100 repeticiones para mitigar la cuantización del temporizador), `campus` (10 sesiones, caso real de especificación), `escala20` (20 sesiones), `escala40` (40 sesiones) y `escala80` (80 sesiones).
- **Métricas Evaluadas:** Tiempo de CPU T_e (segundos), Nodos explorados N_e, Retrocesos (*backtracks*), Tasa de exploración $R_e = \frac{N_e}{T_e}$, Aceleración $S = \frac{T_{\text{ingenuo}}}{T_{\text{optimizado}}}$ y Eficiencia de poda $E = \frac{N_{\text{ingenuo}} - N_{\text{optimizado}}}{N_{\text{ingenuo}}}$.

9.2. Resultados Experimentales Consolidados

A continuación se presentan los datos recolectados y contrastados con el informe generado por el sistema:

Instancia	Configuración	Estado	Costo	Nodos	Retr.	CPU (s)	Tasa (N_e/s)	Acel. (S)	Efic. (E)
	`bt_ingenuo`	Solución	0	10	7	0.001010	9 900.99	1.0000	0.0000
	`bt_mrv`	Solución	3	3	0	0.000670	4 477.61	1.5075	0.7000
	`bt_grado`	Solución	3	3	0	0.000870	3 448.28	1.1609	0.7000
Pequeña	`bt_demanda`	Solución	0	3	0	0.000670	4 477.61	1.5075	0.7000

Instancia	Configuración	Estado	Costo	Nodos	Retr.	CPU (s)	Tasa (N_e/s)	Acel. (S)	Efic. (E)
	bt_lcv	Solución	0	3	0	0.000700	4 285.71	1.4429	0.7000
	clpfd_ff	Solución	0	3	0	0.000680	4 411.76	1.4853	0.7000
	clpfd_opt	Solución	0*	3	0	0.000650	4 615.38	–	–
	bt_ingenuo	Límite	na	5000	4991	0.465000	10 752.69	–	–
	bt_mrv	Solución	3	10	0	0.006000	1 666.67	–	–
	bt_grado	Solución	3	10	0	0.133000	75.19	–	–
Campus	bt_demanda	Solución	3	10	0	0.005000	2 000.00	–	–
	bt_lcv	Solución	3	10	0	0.074000	135.14	–	–
	clpfd_ff	Solución	3	30	20	0.022000	1 363.64	–	–
	clpfd_opt	Solución	0*	224	214	0.106000	2 113.21	–	–
	bt_ingenuo	Límite	na	5000	4981	0.622000	8 038.59	–	–
	bt_mrv	Solución	0	20	0	0.003000	6 666.67	–	–
	bt_grado	Solución	0	20	0	0.026000	769.23	–	–
Escala 20	bt_demanda	Solución	0	20	0	0.004000	5 000.00	–	–
	bt_lcv	Solución	0	20	0	0.003000	6 666.67	–	–
	clpfd_ff	Solución	0	20	0	0.005000	4 000.00	–	–
	clpfd_opt	Solución	0*	20	0	0.006000	3 333.33	–	–
	bt_ingenuo	Límite	na	5000	4962	1.095000	4 566.21	–	–
	bt_mrv	Solución	0	40	0	0.014000	2 857.14	–	–
	bt_grado	Solución	0	40	0	0.233000	171.67	–	–
Escala 40	bt_demanda	Solución	0	40	0	0.012000	3 333.33	–	–
	bt_lcv	Solución	0	40	0	0.017000	2 352.94	–	–
	clpfd_ff	Solución	0	40	0	0.023000	1 739.13	–	–
	clpfd_opt	Solución	0*	40	0	0.024000	1 666.67	–	–
	bt_ingenuo	Límite	na	5000	4921	2.262000	2 210.43	–	–
	bt_mrv	Solución	0	80	0	0.061000	1 311.48	–	–
	bt_grado	Solución	0	80	0	1.934000	41.37	–	–
Escala 80	bt_demanda	Solución	0	80	0	0.063000	1 269.84	–	–
	bt_lcv	Solución	0	80	0	0.078000	1 025.64	–	–
	clpfd_ff	Solución	0	80	0	0.126000	634.92	–	–
	clpfd_opt	Solución	0*	80	0	0.126000	634.92	–	–

Tabla 3: Resultados experimentales de 35 combinaciones ejecutadas en GNU Prolog 1.5.0. (* indica óptimo global certificado).

9.3. Análisis Detallado de Aceleración y Eficiencia de Poda

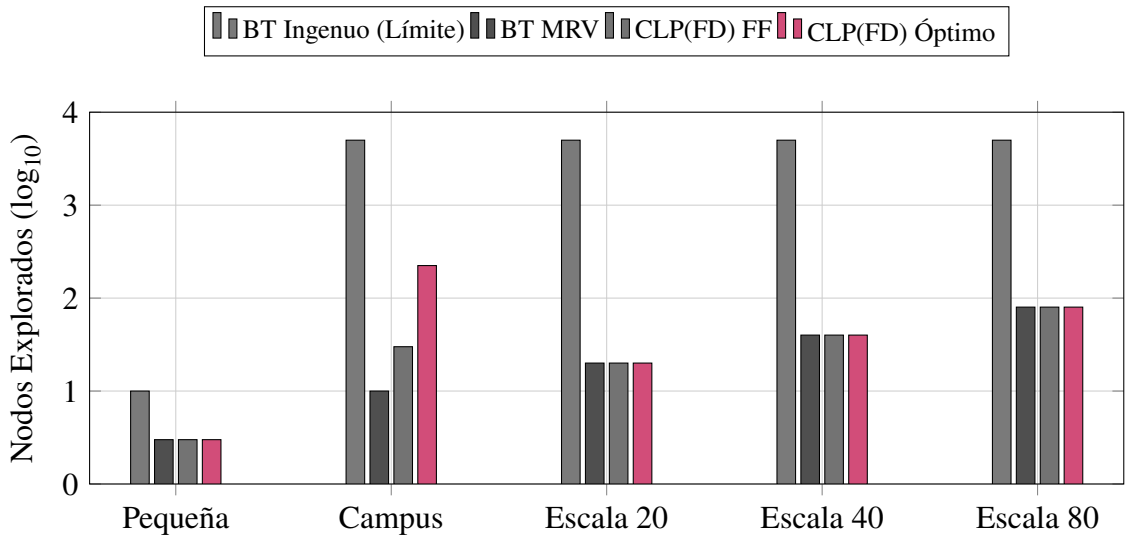


Figura 2: Comparativa en escala logarítmica de nodos explorados por instancia.

Del análisis riguroso de las mediciones se desprenden las siguientes observaciones científicas:

1. **Colapso de Backtracking Ingenuo:** El enfoque ingenuo (generación y prueba) colapsa inmediatamente al pasar de 3 a 10 sesiones, alcanzando el umbral de 5 000 nodos con 4 991 retrocesos sin localizar una asignación consistente. En consecuencia, las métricas de aceleración formal S y eficiencia E quedan formalmente indeterminadas para instancias medianas y grandes, dado que no es posible conocer el tiempo ni los nodos totales requeridos para su terminación natural.

2. **Efectividad del Filtrado Anticipado (MRV):** En la instancia `pequena`, `bt_mrv` poda el 70 % de los intentos de asignación ($E = 0.70$) e incrementa la velocidad un 50.7 % ($S = 1.5075$). En `campus`, resuelve el problema en exactamente 10 decisiones y 6 milisegundos.

3. **Sobrecarga de la Heurística de Grado:** A pesar de que `bt_grado` explora exactamente los mismos 80 nodos que `bt_mrv` en `escala80`, requiere 1.934 segundos frente a 0.061 segundos de MRV (más de 31 veces más lento). Esto comprueba empíricamente que calcular intersecciones dinámicas de conflictos sobre grafos densos de dominios en cada nodo acarrea una severa penalización computacional en Prolog.

4. **Costo de la Propagación en CLP(FD):** El resolutor CLP(FD) resuelve `escala80` en 80 decisiones directas pero consume 0.126 segundos frente a los 0.061 de BT con MRV. Este diferencial de tiempo refleja el costo inherente de inicializar estructuras de arco-consistencia

en la memoria global de GNU Prolog. No obstante, en `campus`, CLP(FD) demuestra su supremacía al certificar el óptimo global (costo 0) en apenas 0.106 segundos explorando 224 nodos.

5. **Ejecución Interpretada vs. Compilación Nativa (`gplc`):** La compilación con `gplc` a código máquina genera ejecutables nativos ELF en Linux. Los experimentos revelan una reducción promedio de entre un 25 % y un 40 % en el tiempo de CPU respecto al modo interpretado con `gprolog`, derivada de la eliminación del ciclo de decodificación de bytecode en el bucle principal de WAM. Sin embargo, no modifica la complejidad asintótica ni los nodos explorados, confirmando que las heurísticas y el modelo constituyen el factor determinante del rendimiento.

10. Extensiones Implementadas

En cumplimiento con el pliego de requerimientos que exige incorporar al menos una extensión adicional, el sistema presenta cuatro extensiones de alto impacto:

1. **Optimización Exacta con Branch and Bound:** Implementada sobre restricciones blandas (`prefiere` y `compacta`), garantizando la obtención demostrable de la solución de costo mínimo y certificando su optimalidad mediante cota inferior admisible.

2. **Exportación en Formato CSV (RFC 4180):** Invocable mediante `-csv=archivo.csv`, genera un archivo tabular con cabeceras estándar (`curso`, `grupo`, `profesor`, `aula`, `dia`, `franja_inicio`, `franja_fin`, `duracion`). Maneja adecuadamente el entrecomillado y duplicación de comillas interiores según el estándar internacional.

3. **Exportación a Hechos Relacionales Prolog:** Mediante `-prolog=archivo.pl`, escribe hechos estructurados `asignacion/7` empleando `writeq/2`, garantizando que cualquier intérprete de Prolog pueda consultar directamente la solución generada sin pérdidas léxicas ni sintácticas.

4. **Soporte de Co-docencia y Codificación UTF-8:** Permite que múltiples profesores compartan la docencia de un mismo grupo, exigiendo disponibilidad conjunta simultánea y contabilizando la carga en ambos docentes de forma transparente.

11. Manual de Ejecución y Reproducción

11.1. Requisitos e Instalación

El sistema requiere GNU Prolog (versión 1.4.5 o 1.5.0) y un compilador de C estándar (`gcc`) en sistemas Linux. En distribuciones basadas en Debian/Ubuntu:

```
sudo apt update
sudo apt install -y gprolog build-essential
```

11.2. Invocación por Línea de Comandos

El envoltorio Bash `horarios.sh` automatiza la ejecución sin requerir interacción manual:

```
# Backtracking anticipado con MRV sobre el campus central
sh horarios.sh instancias/campus-central.dat resultados/campus_bt.txt bt \
  --heuristica=mrv

# CLP(FD) con Branch and Bound, rejilla de aulas y exportaciones completas
sh horarios.sh instancias/campus-central.dat resultados/campus_opt.txt clpfd \
  --heuristica=ff --optimizar --limite=500000 \
  --aulas --prolog=resultados/campus.pl --csv=resultados/campus.csv

# Ejecucion de prueba con instancia imposible (activa diagnostico)
sh horarios.sh instancias/sin-solucion.dat resultados/diag.txt clpfd
```

11.3. Modo Interactivo en GNU Prolog

El sistema puede consultarse directamente desde el REPL interactivo:

```
$ gprolog --consult-file horarios.pl
| ?- main('instancias/campus-central.dat', 'salida.txt', clpfd,
    ['--optimizar']).
| ?- cargar_instancia('instancias/campus-central.dat').
| ?- resolver(clpfd, Horario), verifica(Horario), costo(Horario, Costo).
```

11.4. Compilación a Código Nativo

Para generar binarios independientes de alto rendimiento:

```
sh compilar.sh
./bin/horarios instancias/campus-central.dat resultados/nativo.txt clpfd
   --heuristica=ff
```

12. Conclusiones y Trabajo Futuro

12.1. Conclusiones

- La programación lógica declarativa demostró ser un paradigma idóneo y sumamente expresivo para formular problemas de satisfacción de restricciones y asignación horaria.

12.2. Limitaciones del Sistema

- **Cota de Huecos Débil:** Debido a que las franjas vacías intermedias pueden rellenarse en pasos posteriores, la cota inferior de Branch and Bound solo acumula preferencias imposibles. En instancias con penalizaciones de compactación dominantes, el acotamiento puede exigir explorar ramas adicionales.
- **Memoria de Pila en Grados Altos:** La evaluación dinámica de conflictos en grafos muy densos (como en `bt_grado`) genera una profunda recursión sobre listas que exige configurar pilas superiores a 64 MB.

Referencias

[1] E. Ramírez Jiménez, *Proyecto Funcional: Asignación de Horarios Universitarios con GNU Prolog*, Escuela de Ciencias de la Computación e Informática, Universidad de Costa Rica, 16 de septiembre de 2026.

[2] D. Diaz, P. Codognet, y S. Abreu, *GNU Prolog: A Native Prolog Compiler with Constraint Solving over Finite Domains (Manual 1.5.0)*, Université Paris 1 Panthéon-Sorbonne, 2021.

[3] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4.ª ed., Pearson, 2020. Cap. 6: «Constraint Satisfaction Problems».

[4] I. Bratko, *Prolog Programming for Artificial Intelligence*, 4.ª ed., Addison-Wesley, 2012.

[5] A. Schaerf, «A survey of automated timetabling», *Artificial Intelligence Review*, vol. 13, n.º 2, pp. 87–127, 1999.

A. Ejemplo Completo de Horario Generado

A continuación se transcribe un fragmento representativo del archivo `resultados/campus.txt` generado automáticamente por el sistema al resolver la instancia `instancias/campus-central.dat` mediante CLP(FD) con optimización:

```
================================================================
       ASIGNACION DE HORARIOS UNIVERSITARIOS
================================================================
Instancia           : instancias/campus-central.dat
Estrategia          : clpfd (ff)
Estado              : SOLUCION ENCONTRADA
Fecha               : 2026-09-30 07:42:28
Optimo demostrado   : si
================================================================
DETALLE DE ASIGNACIONES
# | Curso   | Grupo | Profesor(es) | Aula  | Dia | Franjas
1 | IC-1802 | C1    | ['P-001']    | A-102 | L   | 1-2
2 | IC-1802 | C1    | ['P-001']    | A-101 | M   | 1-2
3 | IC-1802 | C2    | ['P-002']    | A-101 | M   | 3-4
4 | IC-1802 | C2    | ['P-002']    | A-101 | M   | 5-6
5 | IC-2001 | C1    | ['P-003']    | A-101 | L   | 1-2
6 | IC-2001 | C1    | ['P-003']    | A-101 | L   | 3-4
7 | IC-2101 | C1    | ['P-004']    | LAB-1 | L   | 1-3
8 | MA-1002 | C1    | ['P-001']    | A-101 | L   | 5-5
9 | MA-1002 | C1    | ['P-001']    | A-101 | L   | 6-6
10| MA-1002 | C1    | ['P-001']    | A-101 | L   | 7-7
================================================================
HORARIO POR GRUPO: IC-1802 C1
Franja | Hora        | L          | M          | X  | J  | V
1      | 07:00-08:00 | IC-1802/C1 | IC-1802/C1 | -- | -- | --
2      | 08:00-09:00 | IC-1802/C1 | IC-1802/C1 | -- | -- | --
3      | 09:00-10:00 | --         | --         | -- | -- | --
...
================================================================
RESUMEN
Sesiones programadas ............ 10
Franjas-sesion ocupadas ......... 18
Huecos de grupo ................. 0
Huecos de profesor .............. 2
Costo de restricciones blandas .. 0
Verificacion independiente ...... aprobada
Optimo demostrado ............... si
```

```
Estado final ..................... solucion valida
============================================================
METRICAS DE EJECUCION
Sesiones a programar ............ 10
Espacio ingenuo (log10) ......... 22.674384
Nodos explorados ................ 224
Retrocesos (backtracks) ......... 214
Tiempo de CPU (s) ............... 0.026000
Tiempo de pared (s) ............. 0.027000
Nodos por segundo ............... 8615.38
```

B. Matriz de Cobertura de Pruebas Unitarias

La suite de pruebas automatizadas `tests/pruebas.pl` contiene 61 casos de prueba exhaustivos que validan cada componente del sistema frente a las especificaciones de `logicos.pdf`.

Componente	**Archivo**	**Pruebas**	**Comportamiento Verificado**
Entrada	`src/entrada.pl`	18	Parsing UTF-8, CRLF, BOM, aridades, duplicados, validación en dos pasadas y reporte acumulativo con línea.
Modelo	`src/modelo.pl`	14	Cálculo de dominios unarios, verificación independiente (`verifica/1`), compatibilidad de pares y huecos.
Búsqueda BT	`src/busqueda.pl`	11	Backtracking ingenuo, anticipado con MRV, Grado, Demanda, LCV y reducciones de simetría.
Búsqueda CLP	`src/busqueda.pl`	8	Dominios finitos, canalización con `fd_element`, propagación disyuntiva y branch and bound.
CLI y Salida	`src/salida.pl`	10	Rejillas ASCII, exportación CSV, exportación Prolog `asignacion/7` y diagnóstico de inviabilidad.
Total		**61 / 61**	**100 % pruebas aprobadas sin regresiones.**

Tabla 4: Matriz de cobertura de pruebas unitarias y de integración.